

Plano de Arquitetura e Estrutura do Aplicativo

1. Visão Geral

O objetivo é desenvolver um aplicativo desktop em Python para gerenciar múltiplas demandas com cronômetros independentes. O aplicativo permitirá o cadastro de demandas, controle de tempo (iniciar, pausar, parar), registro de descrições finais e geração de relatórios.

2. Tecnologias Escolhidas

- **Linguagem de Programação:** Python
- **Interface Gráfica (GUI):** Tkinter (conforme sugestão do usuário)
- **Armazenamento de Dados:** SQLite (banco de dados leve e embutido, ideal para aplicações desktop sem necessidade de servidor externo).

3. Estrutura do Aplicativo (Módulos)

O aplicativo será dividido em módulos para melhor organização e manutenção:

- `main.py` : Ponto de entrada do aplicativo, inicializa a interface gráfica e o controlador.
- `gui.py` : Contém a lógica da interface gráfica (Tkinter), incluindo janelas, widgets e eventos.
- `controller.py` : Atua como intermediário entre a GUI e o modelo de dados, manipulando a lógica de negócios e a interação com o banco de dados.
- `model.py` : Define a estrutura dos dados (Demandas) e a lógica de interação com o banco de dados (SQLite).
- `database.py` : Gerencia a conexão e as operações CRUD (Create, Read, Update, Delete) com o banco de dados SQLite.

4. Estrutura do Banco de Dados (SQLite)

Será criada uma tabela `demands` com a seguinte estrutura:

Campo	Tipo de Dados	Descrição
id	INTEGER	Chave primária, auto-incremento
name	TEXT	Nome da demanda (ex: "Demanda 01")
card	TEXT	Número do card (ex: "CARD-01")
status	TEXT	Status da demanda (em andamento, pausada, finalizada)
accumulated_time	REAL	Tempo total acumulado em segundos
start_time	TEXT	Timestamp de início da última contagem
end_time	TEXT	Timestamp de fim da demanda (após Stop)
description	TEXT	Descrição final do que foi feito

5. Fluxo de Funcionamento

- Inicialização:** O aplicativo carrega as demandas existentes do banco de dados e exibe na interface.
- Cadastro de Demanda:** O usuário insere Nome e Card, e uma nova demanda é criada com status "pausada" e tempo acumulado zero.
- Controle do Cronômetro:**
 - Play:** Inicia o cronômetro para uma demanda. O `start_time` é registrado. Se já estiver em andamento, não faz nada. Se estiver pausada, retoma a contagem a partir do `accumulated_time` existente.
 - Pause:** Pausa o cronômetro. O tempo decorrido desde `start_time` é adicionado ao `accumulated_time`. O `start_time` é resetado. O status muda para "pausada".
 - Stop:** Finaliza o cronômetro. O tempo decorrido é adicionado ao `accumulated_time`. O `end_time` é registrado. O status muda para "finalizada". O campo de descrição é habilitado.
- Multicronômetro:** A lógica de `Play`, `Pause` e `Stop` será independente para cada demanda, garantindo que apenas um cronômetro esteja ativo por vez, mas permitindo alternar entre eles.
- Relatórios:** O usuário seleciona um período de datas. O aplicativo consulta o banco de dados para demandas finalizadas nesse período e exibe os detalhes (Nome, Card, Tempo Total, Descrição).

6. Extensões e Ferramentas

- **IDE:** VS Code (recomendado para desenvolvimento Python).
- **Extensões VS Code:**
 - Python (Microsoft): Suporte completo para Python, incluindo linting, depuração e IntelliSense.
 - Pylance (Microsoft): Extensão de linguagem para Python, oferece recursos avançados de análise de código.
 - SQLite Viewer (ou similar): Para visualizar e gerenciar o banco de dados SQLite durante o desenvolvimento.

Este plano servirá como guia para a implementação do aplicativo.